

同济大学计算机系

人工智能原理与技术课程设计报告

课设（三）

深度学习模块：手写数字识别与垃圾分类



学 号： 2 4 5 1 6 8 8

姓 名： 戴 文 昱

专 业： 计 算 机 科 学 与 技 术

实验框架： MindSpore 2.9.0

实验日期： 2026 年 5 月

目录

1	问题概述	1
1.1	实验背景	1
1.2	实验目标	1
1.3	总体实验流程	1
2	数据集与环境	2
2.1	实验环境	2
2.2	数据集说明	2
2.3	数据预处理	3
3	模型与方法	3
3.1	LeNet-5 模型结构	3
3.2	迁移学习与微调策略	4
3.3	改进模型设计	5
4	代码实现	5
4.1	LeNet-5 实现	5
4.2	预训练参数加载与冻结	6
4.3	改进模型实现	7
4.4	训练循环	8
5	实验结果	9
5.1	MNIST 预训练结果	9
5.2	垃圾分类冻结微调与全量微调	10
5.3	改进模型实验	11
5.4	综合对比	11
6	结果分析	13
6.1	冻结微调与全量微调对比	13
6.2	改进模型效果分析	13
6.3	实验局限	13
7	总结	14

1 问题概述

1.1 实验背景

LeNet-5 与 MNIST 手写数字数据集是卷积神经网络入门实验中最经典的组合。MNIST 图像尺寸小、类别清晰、数据规模适中，适合作为预训练模型的来源。垃圾分类任务同样属于图像分类问题，但它与手写数字识别存在明显差异：垃圾图像通常为彩色自然图像，背景、形状、纹理和拍摄角度更加复杂。若直接从头训练模型，模型容易受到数据量较小、类别差异细微等因素影响。因此，本实验以 LeNet-5 在 MNIST 上的训练结果作为初始模型，并迁移到 26 类垃圾分类任务中，比较不同微调策略的效果。

本实验的重点不是单纯追求最高准确率，而是完成一个完整的深度学习实验流程：数据预处理、模型定义、预训练、参数加载、分类层替换、冻结微调、全量微调、模型改进与结果分析。实验使用 MindSpore 深度学习框架，在 CPU 环境下完成训练和评估。

1.2 实验目标

本实验围绕以下目标展开：

1. 在 MNIST 数据集上训练 LeNet-5，并保存预训练权重。
2. 加载 MNIST 预训练权重，将模型迁移到垃圾分类数据集。
3. 将原始 10 类分类层替换为 26 类垃圾分类层。
4. 对比冻结微调和全量微调两种策略。
5. 设计改进模型，并比较改进前后的准确率、训练时间和内存占用。
6. 通过实验截图、代码截图和结果表格形成可复现的课程设计报告。

1.3 总体实验流程

图 1 给出了本实验的总体流程。首先使用 MNIST 训练 LeNet-5，得到预训练权重；随后加载除最后分类层以外的参数，在垃圾分类数据集上分别进行冻结微调和全量微调；最后构建一个更适合垃圾图片的 RGB 卷积网络，比较其与原始 LeNet-5 迁移方案的差异。

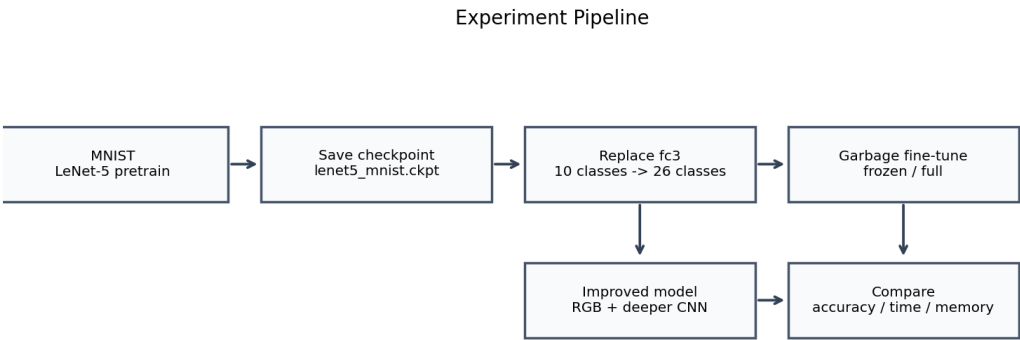


图 1: 实验流程图

2 数据集与环境

2.1 实验环境

本实验使用的主要环境如表 1 所示。

表 1: 实验环境

项目	配置
操作系统	macOS
Python 环境	Conda, Python 3.9
深度学习框架	MindSpore 2.9.0
运行设备	CPU
主要依赖	numpy、Pillow、matplotlib、scikit-learn、psutil

2.2 数据集说明

本实验使用两类数据集：

- MNIST 手写数字数据集：包含 0 到 9 共 10 个类别，用于训练 LeNet-5 预训练模型。
- 垃圾分类数据集：包含 26 个垃圾类别，用于迁移学习微调和改进模型训练。

垃圾分类数据集整理后按 8:2 划分为训练集和验证集。每类训练集约 77 到 85 张图像，验证集约 20 到 22 张图像。部分类别示例如图 2 所示。

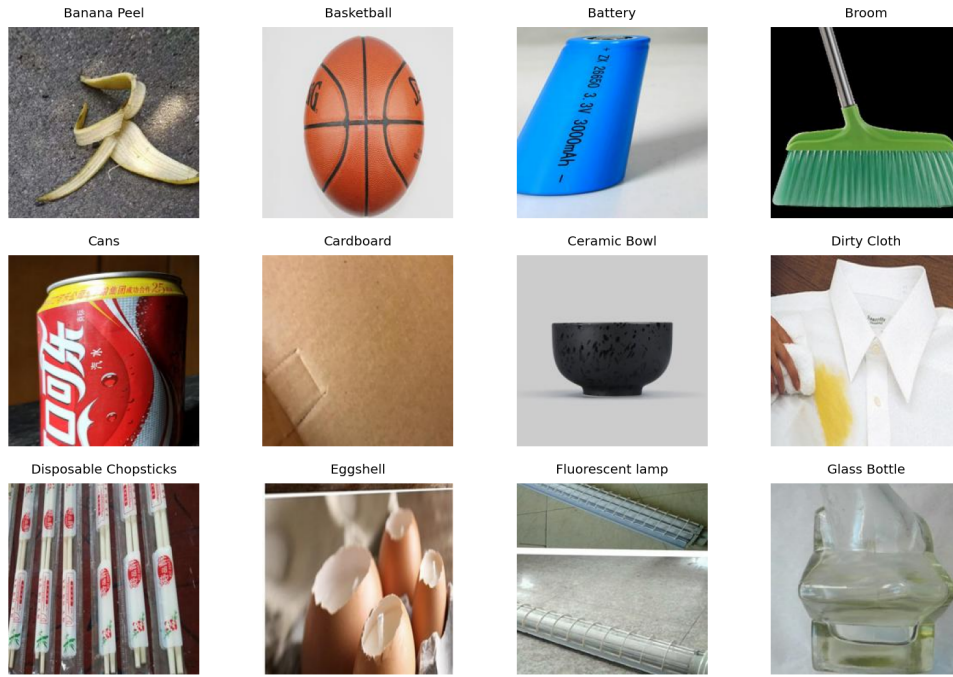


图 2: 垃圾分类数据集部分样本

2.3 数据预处理

为了使 MNIST 预训练模型能够迁移到垃圾分类任务，原始 LeNet-5 迁移实验采用灰度输入。具体处理方式如下：

1. MNIST 图像由 28×28 resize 到 32×32 ，并归一化。
2. 垃圾图片转换为灰度图，resize 到 32×32 。
3. 灰度图输入通道数为 1，与 LeNet-5 的第一层卷积保持一致。

这种处理方式可以保证 MNIST 上训练得到的卷积层参数能够直接迁移，但也会丢失垃圾图片中的颜色信息。因此，在模型改进部分，本实验额外设计了 RGB 输入的改进 CNN，输入尺寸提升为 64×64 ，以保留更多颜色和纹理信息。

3 模型与方法

3.1 LeNet-5 模型结构

LeNet-5 是一种早期经典卷积神经网络，结构简单但具备完整的卷积、池化和全连接分类流程。本实验使用的 LeNet-5 结构如图 3 所示。

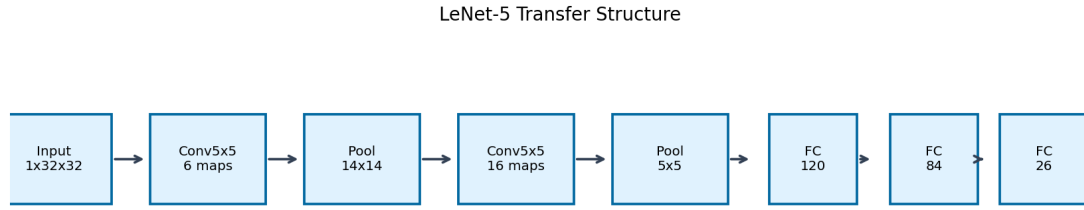


图 3: LeNet-5 迁移到 26 类垃圾分类的结构

网络结构可以概括为：

$$Input \rightarrow Conv1 \rightarrow Pool1 \rightarrow Conv2 \rightarrow Pool2 \rightarrow FC1 \rightarrow FC2 \rightarrow FC3.$$

在 MNIST 预训练阶段，最后的全连接层输出维度为 10；在垃圾分类阶段，最后一层替换为输出维度 26 的分类层。

表 2: LeNet-5 网络结构

层	输入	输出	说明
Conv1	$1 \times 32 \times 32$	$6 \times 28 \times 28$	5×5 卷积
Pool1	$6 \times 28 \times 28$	$6 \times 14 \times 14$	最大池化
Conv2	$6 \times 14 \times 14$	$16 \times 10 \times 10$	5×5 卷积
Pool2	$16 \times 10 \times 10$	$16 \times 5 \times 5$	最大池化
FC1	400	120	全连接层
FC2	120	84	全连接层
FC3	84	10/26	分类层

3.2 迁移学习与微调策略

迁移学习的基本思想是将源任务中学到的参数迁移到目标任务中，利用已有特征提取能力减少目标任务训练成本。本实验源任务为 MNIST 手写数字识别，目标任务为垃圾分类。虽然两个任务差异较大，但卷积层仍然可以提供边缘、轮廓等基础视觉特征。

本实验实现两种微调策略：

- 冻结微调：冻结 conv1、conv2、fc1 和 fc2，只训练新的 fc3 分类层。

- 全量微调：加载预训练参数后，所有层都参与训练。

冻结微调训练参数少，速度快，但适应新任务的能力有限；全量微调可以更新所有层，通常更适合源任务和目标任务差异较大的场景。

3.3 改进模型设计

由于垃圾分类图像是自然图像，颜色和纹理信息比手写数字更重要。原始 LeNet-5 迁移实验为了兼容 MNIST 权重，将垃圾图像转换为灰度 32×32 ，造成明显信息损失。为提升性能，本实验设计了改进模型 ImprovedGarbageCNN：

1. 输入由灰度 32×32 改为 RGB 64×64 。
2. 使用 4 个卷积块，通道数从 32 增加到 128。
3. 每个卷积块加入 BatchNorm，提高训练稳定性。
4. 分类器中加入 Dropout，缓解小数据集上的过拟合。
5. 训练时加入随机裁剪和水平翻转作为数据增强。

4 代码实现

4.1 LeNet-5 实现

图 4 展示了 LeNet-5 的核心代码。该模型在初始化时可以指定输出类别数，因此同一网络既可以用于 MNIST 的 10 类分类，也可以在替换分类层后用于垃圾分类。

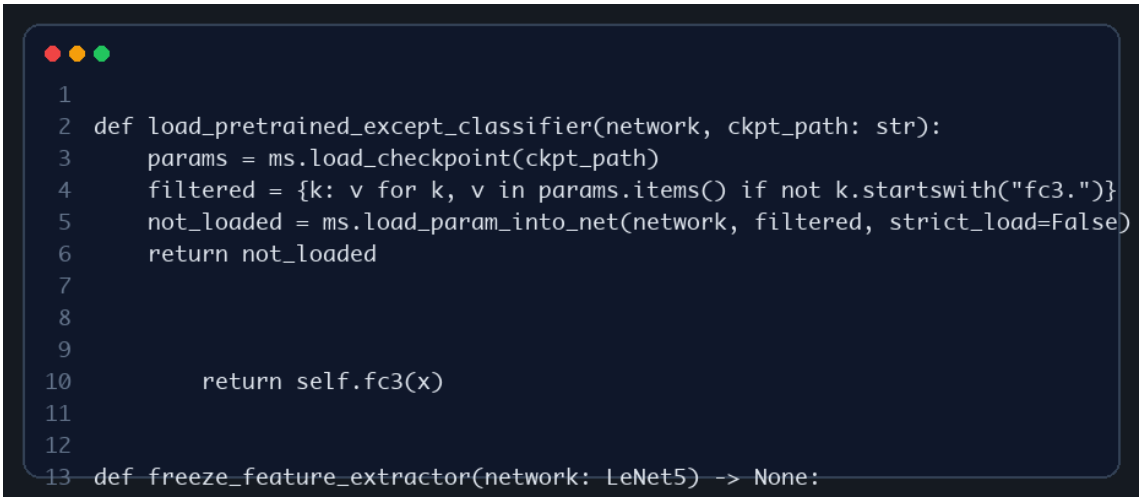
```
1 class LeNet5(nn.Cell):
2     """LeNet-5 for 32x32 grayscale image classification."""
3
4     def __init__(self, num_classes: int = 10):
5         super().__init__()
6         self.conv1 = nn.Conv2d(1, 6, kernel_size=5, pad_mode="valid")
7         self.relu1 = nn.ReLU()
8         self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
9         self.conv2 = nn.Conv2d(6, 16, kernel_size=5, pad_mode="valid")
10        self.relu2 = nn.ReLU()
11        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
12        self.flatten = nn.Flatten()
13        self.fc1 = nn.Dense(16 * 5 * 5, 120)
14        self.relu3 = nn.ReLU()
15        self.fc2 = nn.Dense(120, 84)
16        self.relu4 = nn.ReLU()
17        self.fc3 = nn.Dense(84, num_classes)
18
19    def construct(self, x):
20        x = self.pool1(self.relu1(self.conv1(x)))
21        x = self.pool2(self.relu2(self.conv2(x)))
22        x = self.flatten(x)
23        x = self.relu3(self.fc1(x))
```

图 4: LeNet-5 核心代码截图

代码中 Conv2d 提取局部图像特征, MaxPool2d 降低空间尺寸, 最后通过三层全连接完成分类。输入为 $1 \times 32 \times 32$ 时, 第二次池化后的特征为 $16 \times 5 \times 5$, 因此第一层全连接输入维度为 400。

4.2 预训练参数加载与冻结

迁移到垃圾分类任务时, MNIST 预训练模型最后一层 fc3 的输出维度为 10, 而垃圾分类输出维度为 26, 因此不能直接加载最后分类层参数。本实验过滤掉 fc3.weight 和 fc3.bias, 只加载其余层参数。相关代码如图 5 所示。



```
1
2 def load_pretrained_except_classifier(network, ckpt_path: str):
3     params = ms.load_checkpoint(ckpt_path)
4     filtered = {k: v for k, v in params.items() if not k.startswith("fc3.")}
5     not_loaded = ms.load_param_into_net(network, filtered, strict_load=False)
6     return not_loaded
7
8
9
10     return self.fc3(x)
11
12
13 def freeze_feature_extractor(network: LeNet5) -> None:
```

图 5: 预训练参数加载与冻结代码截图

训练日志中出现 `fc3.weight` 和 `fc3.bias` 未加载的提示是正常现象，原因正是分类层维度发生变化。冻结微调时，前面特征提取层的 `requires_grad` 被设为 `False`，优化器只更新新的分类层。

4.3 改进模型实现

图 6 展示了改进模型的核心结构。与原始 LeNet-5 相比，该模型使用 RGB 输入，并通过更多卷积层提取更丰富的局部特征。

```
1         param.requires_grad = False
2
3
4 class ImprovedGarbageCNN(nn.Cell):
5     """A stronger RGB CNN for garbage classification."""
6
7     def __init__(self, num_classes: int):
8         super().__init__()
9         self.features = nn.SequentialCell(
10             nn.Conv2d(3, 32, kernel_size=3, pad_mode="pad", padding=1),
11             nn.BatchNorm2d(32),
12             nn.ReLU(),
13             nn.MaxPool2d(kernel_size=2, stride=2),
14             nn.Conv2d(32, 64, kernel_size=3, pad_mode="pad", padding=1),
15             nn.BatchNorm2d(64),
16             nn.ReLU(),
17             nn.MaxPool2d(kernel_size=2, stride=2),
18             nn.Conv2d(64, 128, kernel_size=3, pad_mode="pad", padding=1),
19             nn.BatchNorm2d(128),
20             nn.ReLU(),
21             nn.MaxPool2d(kernel_size=2, stride=2),
22             nn.Conv2d(128, 128, kernel_size=3, pad_mode="pad", padding=1),
23             nn.BatchNorm2d(128),
24             nn.ReLU(),
25             nn.MaxPool2d(kernel_size=2, stride=2),
26         )
27         self.flatten = nn.Flatten()
28         self.classifier = nn.SequentialCell(
29             nn.Dense(128 * 4 * 4, 256),
30             nn.ReLU(),
31             nn.Dropout(p=0.3),
32             nn.Dense(256, num_classes),
33         )
34
35     def construct(self, x):
36         x = self.features(x)
```

图 6: 改进模型代码截图

4.4 训练循环

本实验使用交叉熵损失和 Adam 优化器进行训练。每个 epoch 结束后在验证集上计算准确率和损失，同时记录训练时间和内存峰值。图 7 为改进模型训练循环的代码截图。

```
1 optimizer = nn.Adam(network.trainable_params(), learning_rate=args.lr, weight_decay=args.weight_decay)
2 train_cell = nn.TrainOneStepCell(nn.WithLossCell(network, loss_fn), optimizer)
3 train_cell.set_train()
4
5 history = []
6 with Timer() as timer, MemoryTracker() as memory:
7     for epoch in range(1, args.epochs + 1):
8         network.set_train(True)
9         losses = []
10        for images, labels in train_ds.create_tuple_iterator():
11            loss = train_cell(images, labels)
12            losses.append(float(loss.asnumpy()))
13            memory.update()
14        metrics = evaluate(network, val_ds)
15        memory.update()
16        row = {
17            "epoch": epoch,
18            "train_loss": float(np.mean(losses)),
19            "val_loss": metrics["loss"],
20            "val_accuracy": metrics["accuracy"],
21            "memory_peak_mb": memory.peak_mb,
22        }
```

图 7: 训练循环代码截图

5 实验结果

5.1 MNIST 预训练结果

首先在 MNIST 数据集上训练 LeNet-5 两轮，训练日志截图如图 8 所示。最终验证准确率达到 98.31%，说明模型已经具备较好的手写数字识别能力，可作为后续迁移学习的预训练模型。

```
(base) fish37@daiwenyudeMacBook-Air 第三次作业 % cd /Users/fish37/Desktop/人工智能课程/第三次作业
(base) fish37@daiwenyudeMacBook-Air 第三次作业 % conda activate ai_hw3_ms
(ai_hw3_ms) fish37@daiwenyudeMacBook-Air 第三次作业 % python -m src.train_mnist --epochs 2 --batch-size 64
[WARNING] ME(61838:8696029312,MainProcess):2026-05-19-10:30:52.516.709 [mindspore/context.py:1338] For 'context.set_context', the parameter 'device_target' will be deprecated and removed in a future version. Please use the api mindspore.set_device() instead.
[WARNING] ME(61838:8696029312,MainProcess):2026-05-19-10:30:52.678.130 [mindspore/common/_decorator.py:69] 'FusedSparseAdam' is deprecated from version 2.8.0 and will be removed in a future version.
epoch=1 train_loss=0.2081 val_loss=0.0054 val_acc=0.9761
epoch=2 train_loss=0.0630 val_loss=0.0497 val_acc=0.9831
saved checkpoint: checkpoints/lenet5_mnist.ckpt
saved metrics: outputs/mnist_metrics.json
(ai_hw3_ms) fish37@daiwenyudeMacBook-Air 第三次作业 %
```

图 8: MNIST 预训练终端截图

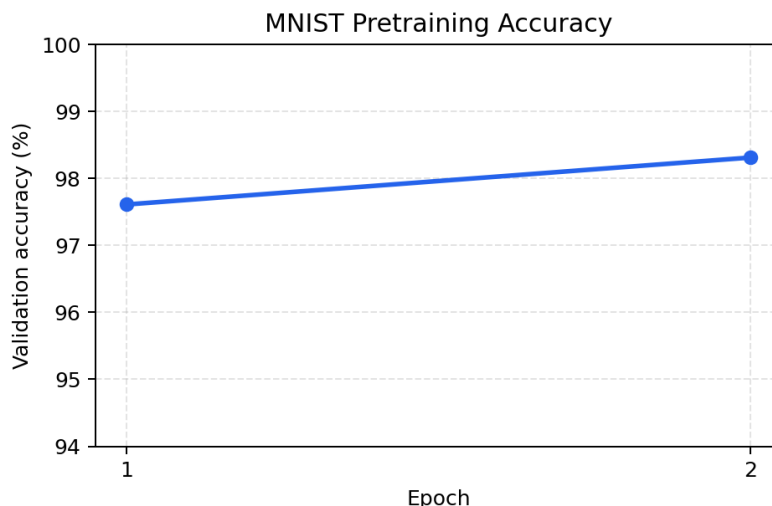


图 9: MNIST 预训练验证准确率

表 3: MNIST 预训练结果

训练轮数	Batch Size	验证准确率	训练时间
2	64	98.31%	28.28 秒

5.2 垃圾分类冻结微调与全量微调

加载 MNIST 预训练权重后，在垃圾分类数据集上分别进行冻结微调 and 全量微调。训练日志截图如图 10 所示。

```

class count: 26
classes: Banana Peel, Basketball, Battery, Broom, Cans, Cardboard, Ceramic Bowl, Dirty Cloth, Disposable Chopsticks, Eggshell, Fluorescent lamp, Glass Bottle, Glassware, Hats, Lighter, Metalware,
Newspaper, Old Mirror, Orange Peel, Paint bucket, Paper, Plastic Bottle, Seashell, Tablet capsules, Toothbrush, Vegetable Leaf
[WARNING] ME(61974:8696029312,MainProcess):2026-05-19-10:34:26.329.773 [mindspore/train/serialization.py:1793] For 'load_param_into_net', 2 parameters in the 'net' are not loaded, because they are
not in the 'parameter_dict', please check whether the network structure is consistent when training and loading checkpoint.
[WARNING] ME(61974:8696029312,MainProcess):2026-05-19-10:34:26.329.824 [mindspore/train/serialization.py:1797] ['fc3.weight', 'fc3.bias'] are not loaded.
[WARNING] ME(61974:8696029312,MainProcess):2026-05-19-10:34:26.331.412 [mindspore/common/_decorator.py:69] 'FusedSparseAdam' is deprecated from version 2.8.0 and will be removed in a future versio
n.
frozen epoch=1 train_loss=3.3301 val_loss=3.2700 val_acc=0.0309
frozen epoch=2 train_loss=3.2604 val_loss=3.2296 val_acc=0.0496
frozen epoch=3 train_loss=3.2145 val_loss=3.2088 val_acc=0.0726
frozen epoch=4 train_loss=3.1781 val_loss=3.1780 val_acc=0.0832
frozen epoch=5 train_loss=3.1478 val_loss=3.1598 val_acc=0.0903
[WARNING] ME(61974:8696029312,MainProcess):2026-05-19-10:34:29.904.029 [mindspore/train/serialization.py:1793] For 'load_param_into_net', 2 parameters in the 'net' are not loaded, because they are
not in the 'parameter_dict', please check whether the network structure is consistent when training and loading checkpoint.
[WARNING] ME(61974:8696029312,MainProcess):2026-05-19-10:34:29.904.111 [mindspore/train/serialization.py:1797] ['fc3.weight', 'fc3.bias'] are not loaded.
full epoch=1 train_loss=3.2466 val_loss=3.1964 val_acc=0.0788
full epoch=2 train_loss=3.1067 val_loss=3.0482 val_acc=0.1115
full epoch=3 train_loss=2.9116 val_loss=2.9358 val_acc=0.1451
full epoch=4 train_loss=2.7530 val_loss=2.8645 val_acc=0.1646
full epoch=5 train_loss=2.6113 val_loss=2.7946 val_acc=0.1912
saved metrics: outputs/garbage_metrics.json

```

图 10: 垃圾分类冻结微调与全量微调终端截图

实验结果表明，全量微调明显优于冻结微调。冻结微调最终准确率为 9.03%，全量微调最终准确率为 19.12%。由于垃圾分类和 MNIST 差异较大，仅训练最后分类层难以充分适应新任务，而全量微调可以更新前面卷积层，因此具有更好的表现。

5.3 改进模型实验

在完成 LeNet-5 迁移学习实验后，本实验进一步训练 RGB 改进模型。训练日志截图如图 11 所示。该模型最终验证准确率达到 47.26%，相较全量微调 LeNet-5 的 19.12% 有明显提升。

```
(ai_hw3_ms) fish37@daiwenyudeMacBook-Air 第三次作业 % python -m src.train_improved_garbage --epochs 5 --batch-size 32 --lr 0.0005
[WARNING] ME(63005:8696029312,MainProcess):2026-05-19-10:55:09.873.550 [mindspore/context.py:1338] For 'context.set_context', the parameter 'device_target' will be deprecated and removed in a future version. Please use the api mindspore.set_device() instead.
[WARNING] ME(63005:8696029312,MainProcess):2026-05-19-10:55:09.920.667 [mindspore/common/_decorator.py:69] 'FusedSparseAdam' is deprecated from version 2.8.0 and will be removed in a future version.
improved epoch=1 train_loss=2.8913 val_loss=2.5116 val_acc=0.2442
improved epoch=2 train_loss=2.4642 val_loss=2.2254 val_acc=0.3416
improved epoch=3 train_loss=2.2105 val_loss=2.0807 val_acc=0.3859
improved epoch=4 train_loss=1.9688 val_loss=1.8635 val_acc=0.4238
improved epoch=5 train_loss=1.7646 val_loss=1.7006 val_acc=0.4726
saved checkpoint: checkpoints/improved_garbage_cnn.ckpt
saved metrics: outputs/improved_garbage_metrics.json
```

图 11: 改进 RGB CNN 训练终端截图

三种垃圾分类实验的准确率变化如图 12 所示。可以看到，冻结微调提升较慢，全量微调有所改善，而改进模型在前几轮即取得更高准确率。

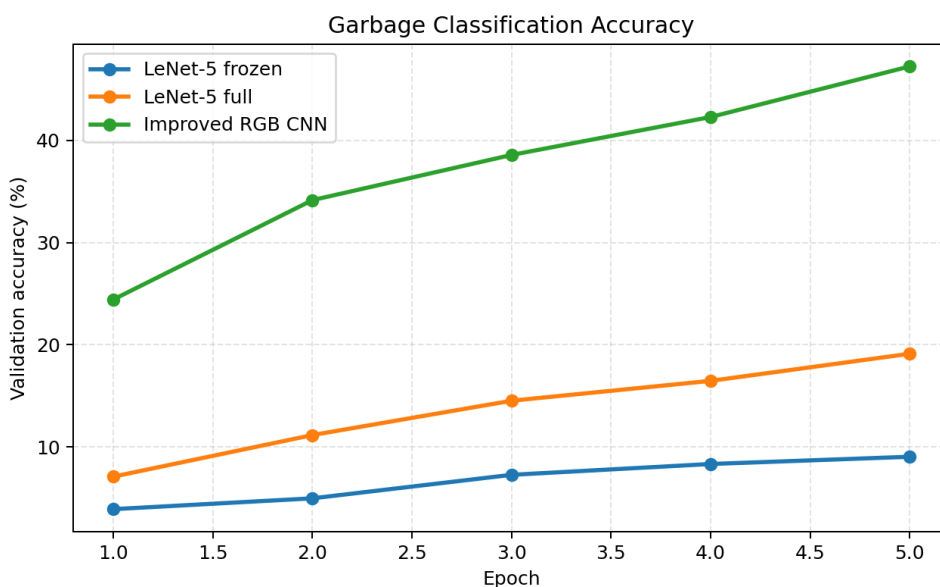


图 12: 垃圾分类验证准确率曲线

5.4 综合对比

表 4 对比了冻结微调、全量微调和改进模型的准确率、训练时间和峰值内存。图 13 以柱状图形式展示了三项指标。

表 4: 垃圾分类实验结果对比

方法	输入	验证准确率	训练时间	峰值内存
冻结微调 LeNet-5	灰度 32×32	9.03%	3.18 秒	398.81 MB
全量微调 LeNet-5	灰度 32×32	19.12%	3.44 秒	408.17 MB
改进 RGB CNN	RGB 64×64	47.26%	65.87 秒	720.89 MB



图 13: 准确率、训练时间和内存占用对比

图 14 为实验结果摘要截图，记录了 MNIST 预训练、垃圾分类微调和改进模型的最终结果。

```
# 实验结果摘要

## MNIST 预训练
- 训练轮数: 2
- 验证准确率: 0.9831
- 训练耗时: 28.28 秒
- 保存权重: `checkpoints/lenet5_mnist.ckpt`

## 垃圾分类微调
- 类别数: 26
- frozen: 验证准确率 0.0903, 训练耗时 3.18 秒, 峰值内存 398.81 MB, 权重 `checkpoints/lenet5_garbage_frozen.ckpt`
- full: 验证准确率 0.1912, 训练耗时 3.44 秒, 峰值内存 408.17 MB, 权重 `checkpoints/lenet5_garbage_full.ckpt`

## 改进模型
- 模型: ImprovedGarbageCNN
- 输入: RGB 64x64
- 验证准确率: 0.4726, 训练耗时 65.87 秒, 峰值内存 720.89 MB
- 保存权重: `checkpoints/improved_garbage_cnn.ckpt`
```

图 14: 实验结果摘要截图

6 结果分析

6.1 冻结微调与全量微调对比

冻结微调只训练最后分类层，因此训练速度最快，内存占用也略低。但由于源任务 MNIST 与目标任务垃圾分类差异较大，MNIST 上学到的灰度手写数字特征并不能充分表示垃圾图像的纹理、颜色和形状特征，导致冻结微调准确率较低。

全量微调允许所有参数更新，模型能够在目标数据集上重新调整卷积核和全连接层，因此准确率从 9.03% 提升到 19.12%。虽然这一结果仍不高，但相对随机猜测 $1/26 \approx 3.85\%$ 已有明显提升，说明迁移学习过程是有效的。

6.2 改进模型效果分析

改进模型准确率达到 47.26%，提升的主要原因包括：

1. RGB 输入保留了垃圾图片的颜色信息，例如塑料瓶、香蕉皮、电池等类别在颜色上有明显差异。
2. 输入尺寸从 32×32 提高到 64×64 ，减少了 resize 造成的细节损失。
3. 更深的卷积网络可以提取更复杂的局部纹理和形状特征。
4. BatchNorm 提升了训练稳定性，Dropout 缓解了小数据集上的过拟合。
5. 随机裁剪和水平翻转提高了模型对拍摄位置和方向变化的适应能力。

代价是训练时间从约 3.44 秒增加到 65.87 秒，峰值内存从 408.17 MB 增加到 720.89 MB。这说明模型改进带来了更高准确率，但也增加了计算资源消耗。

6.3 实验局限

本实验仍存在一些局限：

- LeNet-5 本身较小，且原始迁移实验使用灰度 32×32 输入，不适合复杂自然图像。
- 垃圾分类数据集规模较小，每类样本数量有限，模型泛化能力容易受限。
- 本实验未使用 ImageNet 上预训练的 MobileNetV2 或 ResNet，因此改进模型仍是从头训练。
- LoRA 更适用于大规模线性层或 Transformer 注意力层。本实验主体为小型 CNN，因此主要比较冻结微调和全量微调，并将模型结构改进作为进一步优化方向。

7 总结

本实验完成了从 MNIST 手写数字识别到垃圾分类任务的迁移学习流程。首先使用 LeNet-5 在 MNIST 数据集上进行预训练，并取得 98.31% 的验证准确率；随后加载预训练权重，将最后分类层替换为 26 类输出，在垃圾分类数据集上完成冻结微调 and 全量微调。实验结果表明，全量微调优于冻结微调，说明当源任务和目标任务差异较大时，更新全部网络参数更有利于模型适应目标数据。

在模型改进部分，本实验设计了 RGB 输入的更深 CNN，加入 BatchNorm、Dropout 和数据增强。改进模型将垃圾分类验证准确率提升到 47.26%，显著高于原始 LeNet-5 的迁移学习结果，但训练时间和内存占用也明显增加。整体来看，本实验较完整地覆盖了预训练模型加载、迁移学习、冻结微调、全量微调、模型改进和实验对比分析等深度学习实践内容。

参考资料

- [1] MindSpore 官方文档: <https://www.mindspore.cn/>
- [2] MindSpore Course: <https://gitee.com/mindspore/course>
- [3] LeCun Y., Bottou L., Bengio Y., Haffner P. Gradient-Based Learning Applied to Document Recognition.
- [4] 课程提供的垃圾分类数据集与实验说明。